

04 BEU Platform Integration Test Plan

- [BEU — SMC Platform Integration & Test Plan](#)
 - [1. What “SMC platform test” means here](#)
 - [2. Current state \(as of this writing\)](#)
 - [3. Address-map note \(do not get confused\)](#)
 - [4. Work breakdown \(what needs to be done\)](#)
 - [Phase A — Merge / land the unit model](#)
 - [Phase B — Replace `stub\_beu` in the platform \(required before any VP test\)](#)
 - [Phase C — Interrupt wiring \(needed for IRQ platform tests\)](#)
 - [Phase D — Error injection from the platform \(firmware cannot call C++\)](#)
 - [Phase E — Firmware test: `sw/smc-vp-tests/smc-beu-test/`](#)
 - [5. Suggested implementation order \(for planning\)](#)
 - [6. Explicit non-goals \(for this platform test\)](#)
 - [7. Checklist \(copy into the PR\)](#)
 - [8. One-sentence summary for your manager](#)

BEU — SMC Platform Integration & Test Plan

Audience: manager + implementer.

Goal: explain what is still missing to run a **platform-level** BEU test on `smc-vp` (firmware on the CVA6 cluster), and the concrete work items to close that gap.

Related: unit model + benches live in `smc/peripherals/beu/` (branch `feature/beu`); this doc is only about **platform integration + VP firmware tests**.

1. What “SMC platform test” means here

There are two different test layers:

Layer	Where	What it proves
Unit (standalone)	<code>smc/peripherals/beu/run_tests.sh</code>	Model register semantics, first-error latch, accrued bits, IRQ equations — driven by a C++ TB via TLM + <code>inject_error()</code>
Platform (system)	<code>sw/smc-vp-tests/smc-beu-test/</code> on <code>smc-vp</code>	Firmware on the real cluster reaches BEU through the fabric, programs registers, and (optionally) sees interrupt delivery like production SW

Your manager's ask is the **platform** row. Unit tests alone do **not** satisfy it (see `.cursor/rules/new-ip-ci-checklist.mdc`: new IPs need a VP / platform regression once they are reachable from the platform).

2. Current state (as of this writing)

Item	Status
SystemC BEU LT model (<code>smc::beu</code>)	Done on <code>feature/beu</code> (unit tests + docs + PDFs)
Registered in <code>smc/run_all_smc_tests.sh</code>	Done on <code>feature/beu</code>
Wired into <code>smc-vp / smc_platform</code>	Not done — still <code>stub_beu</code>
Linked in <code>vp/platform/smc/CMakeLists.txt</code>	Not done — no <code>smc_beu</code>
Firmware test <code>sw/smc-vp-tests/smc-beu-test/</code>	Missing
CI <code>smc-vp</code> job picks up new <code>smc-*</code> dirs automatically	Yes (directory discovery) — once the test dir exists

Evidence on `main` today:

- Platform still instantiates a stub:

```
stub_target<64> stub_beu{"stub_beu"};
```

- Front-port route binds that stub at the **local alias** base:

```
static constexpr uint64_t A_BEU = 0xC0C1'0000ULL;
```

```
front_port_router.add_route(6, A_BEU, 0x10000, "beu");
```

```
front_port_router.out[6].bind(stub_beu.reg_socket);
```

- Firmware header already has the alias constant:

```
#define SMC_BEU_BASE 0xC0C10000ULL
```

- Integration guide still says the model is missing:

```
|`0xC801_0000 + N*0x1000`
|4 x `0x30`
|Stub/model needed
|RTL connects per-core BEU registers. No SystemC BEU model exists yet.
```

So: **you cannot write a meaningful platform BEU test until the stub is replaced by the real model in `smc_platform`**. Against the stub, writes are ignored and reads return 0 — any firmware check of `ENABLE` reset (`0xE6`) will fail.

3. Address-map note (do not get confused)

Two address views exist; both are intentional:

View	Base	Notes
HW / RDL / memmap	<code>0xC801_0000 + N*0x1000</code>	Per-core 4 KiB window (N = 0..3)
Platform local alias (what firmware uses)	<code>0xC0C1_0000 + N*0x1000</code>	Same layout, remapped into the cluster MMIO alias aperture [<code>0xC000_0000</code> , <code>0xC100_0000</code>)

Firmware and `smc_common.h` must keep using `SMC_BEU_BASE = 0xC0C10000`.

The model's per-instance window size remains `0x1000`; the platform currently routes a **64 KiB** blob (`0x10000`) to one stub — that must become **four** `smc::beu` instances (or an `addr_router` with four outputs).

4. Work breakdown (what needs to be done)

Phase A — Merge / land the unit model

1. Land `feature/beu` on `main` (or rebase onto current `main` first).
2. While merging, align with the shared AXI extension rule:
 - Prefer `#include "smc_axi_extension.h"` from `smc/common/include`
 - Drop the local `smc/peripherals/beu/include/smc_tlm_extensions.h` copy if `main` already uses the shared header (other IPs already do).

3. Keep unit gates green: `./run_tests.sh, --asan, --coverage (≥ 95%)`.

Exit criteria: `smc_beu` library builds from `smc/peripherals/beu` on `main`.

Phase B — Replace `stub_beu` in the platform (required before any VP test)

Status: DONE — `smc::beu[0..3]` bound via `beu_router`; `stub_beu` removed; `smc_beu` linked into `smc-vp`; IRQ outputs parked on `beu_irq_local` / `beu_irq_plic` sinks until Phase C.

Files to touch:

File	Change
<code>vp/platform/smc/CMakeLists.txt</code>	<code>add_subdirectory(.../beu) + target_link_libraries(smc-vp PRIVATE smc_beu)</code>
<code>vp/platform/smc/smc_platform.hpp</code>	<code>#include "beu.h";</code> replace <code>stub_beu</code> with <code>sc_vector<beu> beu_{"beu", NUM_HARTS}</code> (or 4 named instances)
<code>vp/platform/smc/src/smc_platform.cpp</code>	Unbind <code>stub</code> ; route <code>A_BEU + N*0x1000</code> → <code>beu_[N].reg_socket</code> ; bind <code>rst_n_i</code> ; wire IRQs (see below)
<code>smc/doc/systemmc_tlm2_integration_guide.adoc</code>	Update “Stub/model needed” → real model
<code>vp/platform/smc/inc/stub_target.h</code> comment	Remove BEU from “unmodeled” list

Routing detail: today one route covers `0xC0C1_0000` size `0x10000`. Prefer:

- expand `front_port_router` outputs, **or**
- insert a small `addr_router<64,64>` behind `out[6]` with 4 child routes of size `0x1000`.

BEU is **64-bit** (`regwidth=64`); the front-port path is already 64-bit → **no** `width_adapter` needed (unlike PLIC/CLINT).

Exit criteria: rebuild `smc-vp`; a simple firmware peek of `*(volatile uint64_t*)(SMC_BEU_BASE + 0x10)` returns reset `ENABLE == 0xE6`.

Phase C — Interrupt wiring (needed for IRQ platform tests)

Status: C.1 DONE — `smc_cpu_cluster::beu_nmi_in[i]` added; rising edge calls `inject_nmi(i)`. Platform binds `beu_[n].irq_local_o` → `beu_irq_local[n]` → `cluster.beu_nmi_in[n]`.

Status: C.2 DEFERRED — OCA `interrupts.adoc` / `cpu_interrupts.adoc` map BEU to the per-tile buserror/NMI path, not a PLIC source ID. Rocket `irq_plic_o` remains bound to sink signals until a PLIC source is documented in RTL.

The BEU model exposes two outputs per instance:

- `irq_local_o` — NMI-like, **bypasses PLIC** (HW: `interrupts.adoc`)
- `irq_plic_o` — optional PLIC path (Rocket `PLIC_ENABLE`; maskable)

C.1 Local / NMI path (primary HW path)

Today the cluster has:

- ports: `irq_sw` / `irq_timer` / `irq_ext` only
- API: `cluster.inject_nmi(hart, cause)` (debug / TB hook — **not** a SystemC port)

So `beu_[n].irq_local_o` cannot be bound “as-is” to the cluster yet.

Required platform/cluster work (pick one):

- Preferred:** add a per-hart `sc_in<bool> irq_nmi` (or reuse a dedicated local line) on `smc_cpu_cluster` that calls the ISS NMI path on rising edge; bind `beu_[n].irq_local_o → cluster.irq_nmi[n]`.
- Interim:** platform `SC_METHOD` sensitive to each `irq_local_o` that calls `cluster.inject_nmi(n, cause)` when asserted (cause from BEU `CAUSE` via backdoor or a fixed test cause).

Until this is done, firmware **cannot** prove the local interrupt path end-to-end.

C.2 PLIC path (optional / secondary)

- Extend `interrupt_aggregator` inputs: add 4 BEU PLIC lines.
- Map them to the correct PLIC source bits from the HW interrupt map (`cpu_interrupts.adoc` / RTL).
(Confirm IDs with RTL — do not invent.)
- Bind `beu_[n].irq_plic_o → intagg.src[...]`.

Exit criteria: with `LOCAL_ENABLE` / `PLIC_ENABLE` programmed, an injected error asserts the expected line into the cluster (NMI and/or PLIC MEIP).

Phase D — Error injection from the platform (firmware cannot call C++)

Status: D1 DONE — `smc_platform` exposes three CCI-driven injection slots

(`beu_inject{0,1,2}_{enable,core,src,addr}`, declared in `smc_platform.hpp` / initialized in `smc_platform.cpp`). Each slot, if `enable=true`, calls `beu_[core].inject_error(src, addr)` once at elaboration (before `sc_start()`), so the resulting `ACCRUED` / `CAUSE` / `PHYS_ADDR` state is already latched by the time firmware’s first register read runs. All slots default to `enable=false`, so every other `smc-vp` invocation (including plain `smc-beu-test`) is unaffected; `smc-beu-error-test/smc_beu_error_test.ini` opts in explicitly. D2 (magic-MMIO/scratch handshake) was not needed once D1 covered the E2/E3 test cases below.

Unit tests call `dut.inject_error(src, addr)`. Firmware has **no** natural way to create a cache/TileLink ECC event in the VP (those sources are not modeled as live hardware inputs yet).

Pick an injection strategy for platform tests:

Option	Pros	Cons	Recommendation
D1. Host hook in <code>smc-vp</code> / platform — e.g. CCI flag or “after ELF load, inject once”	Clean; mirrors TB	Not firmware-driven	Good for CI smoke of IRQ path
D2. Magic MMIO / scratch handshake — firmware writes a scratchpad word; platform <code>SC_METHOD</code> sees it and calls <code>beu_[0].inject_error(...)</code>	Fully firmware-orchestrated	Test-only side channel	Good for <code>smc-beu-test</code>
D3. Register-only test — no injection; only RW/reset/mask checks	Smallest change	Does not cover latch / IRQ	Acceptable as Phase E1 only

Recommend: **E1 register smoke first**, then **D2** for a full service routine test.

Phase E — Firmware test: `sw/smc-vp-tests/smc-beu-test/`

Follow the existing pattern (`smc-dma-test`, `smc-i2c-loopback-test`, README “Writing a New Test”).

E.1 Directory layout

```
sw/smc-vp-tests/smc-beu-test/
Makefile      # SRCS = start.S + main.c + printf.c; include ../Makefile.common
main.c        # firmware under test
```

`run_smc_vp_tests.sh` auto-discovers `smc-*/` — no runner edit required.

E.2 Add register helpers to `smc_common.h`

```
/* Per-core BEU (local alias). N = 0..3 */
#define SMC_BEU_BASE_N(n) (SMC_BEU_BASE + 0x1000ULL * (n))

#define BEU_CAUSE          0x00u
#define BEU_PHYS_ADDR      0x08u
#define BEU_ENABLE         0x10u
#define BEU_PLIC_ENABLE    0x18u
#define BEU_ACCRUED        0x20u /* ACCRUED_ENABLE in RDL */
#define BEU_LOCAL_ENABLE   0x28u

#define BEU_VALID_MASK     0xE6ull
#define BEU_SRC_DEU        (1ull << 7) /* example source bit */
```

Use 64-bit accessors (BEU is 64-bit). Today `REG_READ` / `REG_WRITE` are 32-bit — add `REG_READ64` / `REG_WRITE64` (or two 32-bit halves if the fabric path splits; prefer true 64-bit if the MMIO path supports it, matching the model).

E.3 Suggested test cases (firmware)

E1 — Register smoke (minimum viable platform test)

1. Read `ENABLE @ hart0 BEU` → expect `0xE6`.
2. Write `PLIC_ENABLE / LOCAL_ENABLE` with `0xFF`; read back → `0xE6` (mask).
3. Write `PHYS_ADDR` → ignored; read still previous/0.
4. Write `CAUSE = 0` while clear → stays 0.
5. Print `PASS / FAIL` over UART0 (same convention as other tests).

E2 — First-error + accrued + SW ack — DONE (`smc-beu-error-test/`)

Implemented via the D1 hook instead of a live-during-firmware inject (avoids racing/timing the injection against firmware execution): two errors are injected into BEU0 at elaboration (`DCACHE_UNCORRECTABLE` then `DCACHE_TLBUS`), and firmware asserts the resulting state:

1. ~~Program `LOCAL_ENABLE`~~ — not needed for this pass; `ENABLE` (recording) defaults to all-on, so both injected sources latch/accrue without firmware touching any mask register (see the “NMI/PLIC” note below).
2. Trigger inject — done by the platform’s D1 hook before `sc_start()`.
3. Read `ACCRUED` → both source bits set; `CAUSE` → 7 (`DCACHE_UNCORRECTABLE`, the first injected+enabled error); `PHYS_ADDR` → the first injection’s address.
4. Second inject of another source (`DCACHE_TLBUS`) → `ACCRUED` accumulates both bits; `CAUSE` holds the first (7), confirmed by firmware.
5. Write `ACCRUED_ENABLE = 0`, `CAUSE = 0` (ISR ack / re-arm) → firmware reads both back as 0.
6. NMI/PLIC delivery is **intentionally not exercised**: `LOCAL_ENABLE / PLIC_ENABLE` are left at their reset value (0) throughout, so the already-accrued errors never assert `irq_local_o/irq_plic_o` into the running hart. Proving end-to-end NMI control transfer (trap vector entry, `mnstatus/mnepc` save-restore, safely resuming firmware afterwards) needs a dedicated NMI-vector handshake with the cluster and is tracked as a follow-up (see §5 item 9 below).

E3 — Per-core isolation — DONE (`smc-beu-error-test/`, combined with E2)

- A third injection targets BEU1 only (`ICACHE_TLBUS`); firmware confirms BEU1 shows exactly that error and BEU2/BEU3 (no injection at all) read back fully clear (`ACCRUED / CAUSE / PHYS_ADDR` all 0) — proves `beu_router` demuxes all `NUM_BEU` windows independently and that injecting into one core’s model never leaks into another.

E.4 Pass criteria

- UART output contains `PASS` and not `FAIL` (same as other `smc-vp-tests`).
 - `./run_smc_vp_tests.sh smc-beu-test` and `./run_smc_vp_tests.sh smc-beu-error-test` both exit 0.
 - CI `smc-vp` job green with the new directories present.
-

5. Suggested implementation order (for planning)

1. Land feature/beu on main	[Phase A]	DONE
2. CMake + replace stub with 4× beu	[Phase B]	DONE
3. Firmware E1 register smoke test	[Phase E1]	DONE
4. Injection hook (CCI, elaboration-time)	[Phase D1]	DONE
5. Wire irq_local → cluster NMI	[Phase C.1]	DONE
6. Firmware E2 accrual/first-error/SW-ack test	[Phase E2]	DONE
7. Firmware E3 per-core isolation test	[Phase E3]	DONE
8. Update integration guide + smc-vp-tests README	[remaining]	
9. (Follow-up) End-to-end NMI delivery firmware test	[Phase E2-NMI]	open
10. (Optional) PLIC path (needs an HW source ID)	[Phase C.2]	deferred

Estimate guidance (rough):

Phase	Rough effort	Depends on
A	small (merge/CI)	PR review
B	medium (router + bind + rebuild VP)	A
E1	small	B
D1 + E2 + E3	small–medium (CCI hook is much lighter than a live scratch/MMIO handshake)	B, C.1
E2-NMI follow-up	medium (needs a real NMI-vector handshake in firmware)	E2
C.2	small–medium	HW source ID assignment (blocked on RTL)

6. Explicit non-goals (for this platform test)

- Bit-accurate SCL/SDA-style modeling — N/A for BEU.
- Modeling real cache ECC / TileLink bus monitors inside Whisper — out of scope; `inject_error()` is the agreed LT abstraction.
- Replacing unit ASan/coverage gates — platform test **adds** system coverage; it does not replace `./run_tests.sh --asan / --coverage` on the IP.

7. Checklist (copy into the PR)

- ☒ `smc_beu` linked into `smc-vp`
- ☒ `stub_beu` removed; 4 per-core windows at `0xC0C1_0000 + N*0x1000`
- ☒ `rst_n_i` bound on each BEU
- ☒ `smc_common.h` BEU offsets + 64-bit helpers + `beu_src` bit-position macros
- ☒ `sw/smc-vp-tests/smc-beu-test/` with Makefile + `main.c` (E1 register smoke)
- ☒ `sw/smc-vp-tests/smc-beu-error-test/` with Makefile + `main.c` + custom `.ini` (E2 accrual/first-error/SW-ack + E3 per-core isolation, driven by the platform's D1 CCI error-injection hook)
- ☒ `./run_smc_vp_tests.sh smc-beu-test` → PASS
- ☒ `./run_smc_vp_tests.sh smc-beu-error-test` → PASS
- ☒ Local NMI path wired (`beu_nmi_in` ← `irq_local_o`); PLIC path deferred (no HW source ID)

- ☒ Integration guide + stub comment updated (README Available Tests still pending Phase E)
 - ☐ CI `smc-vp` job includes the new tests via directory discovery (should be automatic — `run_smc_vp_tests.sh` globs `smc-*/` with a `Makefile` — but not yet confirmed against a live CI run)
 - ☐ (Follow-up, not required for this PR) end-to-end NMI delivery firmware test: firmware installs an NMI-vector handler, enables `LOCAL_ENABLE` for an already-accrued source, and confirms the handler ran and firmware resumed cleanly afterwards
-

8. One-sentence summary for your manager

Unit BEU is done; platform integration is done through Phase E3 — the real per-core BEU is bound into `smc-vp`, the local (NMI-like) line is wired to the cluster, and two firmware tests (`smc-beu-test` register smoke, `smc-beu-error-test` error-injection/accrual/SW-ack/per-core-isolation, driven by a CCI-based test-only injection hook) both pass; only end-to-end NMI control-transfer and the optional PLIC source-ID mapping remain open.